

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

CO2 Assessment Tool 2 — Architecture Design Assignment

SOFTWARE ARCHITECTURE DESIGN REPORT

for

AI-Based Smart Healthcare Appointment and Patient Flow Optimization Platform

Field	Details
Course Code / Title	CSA10 – Software Engineering
CO Assessed	CO2 – Perform detailed requirement analysis, design scalable architectures, and prioritize features with a focus on cross-disciplinary skills
Assessment Tool	Assessment Tool 2 – Architecture Design Assignment
Weightage / Total Marks	20% / 25
Student Name	DK Hari Krishna
Register Number	192321176
Date	28/08/26
Duration	60 minutes

Code of Conduct: I certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

Table of Contents

1. Introduction 3

2. System Requirement Analysis 4

3. Selection of Software Architecture 6

4. Software Architecture Diagram 8

5. Components and Their Interactions 10

5A. Component Interface Specifications 12

5B. Deployment View 13

5C. Architectural Risk Assessment 14

6. Quality Attribute Analysis 15

7. Architectural Justification and Trade-offs 16

8. Conclusion 17

1. Introduction

1.1 Purpose

This report presents the software architecture design for the AI-Based Smart Healthcare Appointment and Patient Flow Optimization Platform. It analyzes the requirements that influence the architecture, selects and justifies an appropriate architectural style, presents a complete architecture diagram, describes each component and its interactions, and evaluates the design against key quality attributes. This report has been prepared as part of the Architecture Design Assignment (CO2 – CSA10 Software Engineering).

1.2 System Overview

The platform helps hospitals manage appointment scheduling and patient flow using AI-based prediction, automated resource allocation, telemedicine support, real-time notifications, and performance analytics. It serves patients, doctors, receptionists, and hospital administrators, and integrates with external systems such as SMS/email gateways, video-conferencing platforms, hospital information systems (HIS/EHR), and payment gateways.

1.3 Scope of Architecture

The architecture covers the client-facing applications, the gateway and security layer, the core business microservices, asynchronous event handling, the data layer, integration with external systems, and cross-cutting operational concerns such as logging, monitoring, and deployment. Clinical decision-making and inpatient/ward management are outside the scope of this architecture.

2. System Requirement Analysis

The problem statement requires a system that predicts patient arrivals, optimizes doctor and resource allocation, prioritizes emergency cases, supports telemedicine, and sends real-time notifications, while generating hospital performance analytics. These objectives directly shape the architectural decisions described in Section 3.

2.1 Functional Requirements Influencing Architecture

Requirement Area	Architectural Implication
Appointment booking, rescheduling, and cancellation	Requires a dedicated Appointment&Scheduling component with its own data store, independently deployable from other concerns.
AI-based prediction of patient arrivals and schedule optimization	Requires a separable, compute-intensive AI/ML service that can scale independently and be retrained without affecting core scheduling logic.
Doctor and resource allocation	Requires a Resource Allocation component that consumes AI predictions and real-time queue data to recommend allocations.
Telemedicine consultations	Requires integration with an external video-conferencing API through a dedicated Telemedicine service.
Real-time notifications	Requires an asynchronous, event-driven communication mechanism rather than tight synchronous coupling, so notification delivery does not block core transactions.
Emergency case prioritization	Requires low-latency event propagation from the queue/appointment component to notification and resource-allocation components.
Hospital performance analytics	Requires an Analytics service with access to aggregated data separate from live transactional data, to avoid impacting operational performance.

2.2 Non-Functional Requirements Influencing Architecture

Quality Attribute	Architectural Implication
Scalability	Independent services must scale horizontally to handle growth in hospitals, departments, and concurrent users, favoring a distributed, loosely coupled architecture.
Performance	AI recommendations and notifications must be produced within a few seconds, favoring caching, asynchronous processing, and horizontally scaled compute for the prediction service.
Reliability	Failure of one component (e.g., analytics) must not bring down critical functions (e.g., booking), favoring isolated services with independent failure domains.
Availability	24/7 operation requires redundancy, health checks, and automated failover across all layers.
Security	Role-based access for patients, doctors, receptionists, and administrators, plus protection of sensitive health data, favoring a centralized gateway for authentication and consistent policy enforcement.
Maintainability	Frequent feature changes (e.g., new notification channels) should be isolated to a

Quality Attribute	Architectural Implication
	single service, favoring modular, independently deployable components.
Interoperability	Integration with external HIS/EHR systems requires a dedicated integration component that isolates external protocol complexity (HL7/FHIR) from the rest of the system.

2.3Key Quality Attributes Summary

Based on the above analysis, the architecture must prioritize scalability (to handle growing hospital branches and concurrent users), maintainability (to support frequent feature evolution such as new AI models or notification channels), reliability and availability (since the system supports time-critical healthcare operations), and security (to protect sensitive patient data). Performance is also critical given the real-time nature of predictions, notifications, and queue updates.

3. Selection of Software Architecture

3.1 Candidate Architectural Styles Considered

Style	Suitability for This System
Layered Architecture (N-Tier)	Simple and well understood, but a single deployable layered application would tightly couple the AI prediction workload with core scheduling, making independent scaling and updates difficult.
Monolithic Client-Server	Easier to build initially, but as features (telemedicine, analytics, AI) grow, the codebase becomes harder to maintain and scale, and a single failure can affect the entire system.
Microservices Architecture	Allows each concern (scheduling, AI prediction, notifications, telemedicine, analytics, EHR integration) to be developed, deployed, and scaled independently, matching the system's diverse and evolving requirements.
Pure Event-Driven Architecture	Well suited to asynchronous notification and prediction workflows, but using it exclusively (with no synchronous APIs) would complicate simple request/response interactions such as booking confirmation.
Serverless Architecture	Attractive for unpredictable, bursty workloads (e.g., notification fan-out), but less suitable as the sole style for stateful, latency-sensitive core scheduling logic and long-running AI model serving.

3.2 Selected Architecture

This report selects a Hybrid Microservices Architecture with an API Gateway and Event-Driven communication between services, layered internally within each service. Core business capabilities (authentication, appointment scheduling, AI prediction, resource allocation, telemedicine, notifications, analytics, and EHR integration) are implemented as independent microservices, exposed through a single API Gateway, and coordinated using both synchronous REST calls (for direct request/response operations, e.g., booking) and an asynchronous event bus (for cross-cutting workflows, e.g., emergency alerts and notifications).

3.3 Justification

- Independent scalability: The AI Prediction&Optimization service is compute-intensive and can be scaled independently of lighter services such as Authentication, avoiding wasted resources.
- Fault isolation: A failure or slowdown in the Analytics service does not affect the ability of patients to book appointments, since each service has an independent failure domain.
- Technology flexibility: The AI Prediction service can use Python-based ML frameworks while other services use different technology stacks best suited to their responsibility, without forcing a single technology choice on the whole system.
- Alignment with functional boundaries: Each microservice maps directly to a cohesive business capability identified in Section 2.1, which simplifies ownership, testing, and maintenance.

- Support for real-time behaviour: The event bus enables asynchronous, low-latency propagation of events such as emergency-case flags and notification triggers, which a purely synchronous layered design would handle less efficiently.
- Suitability for phased delivery: Individual services (e.g., Telemedicine, Analytics) can be added or upgraded after the initial release without redesigning the whole system, supporting the project's phased/academic timeline.

4. Software Architecture Diagram

The diagram below illustrates the complete architecture: the client layer, the API gateway and security layer, the core microservices layer, the event bus, the data layer, external system integrations, and cross-cutting operational concerns.

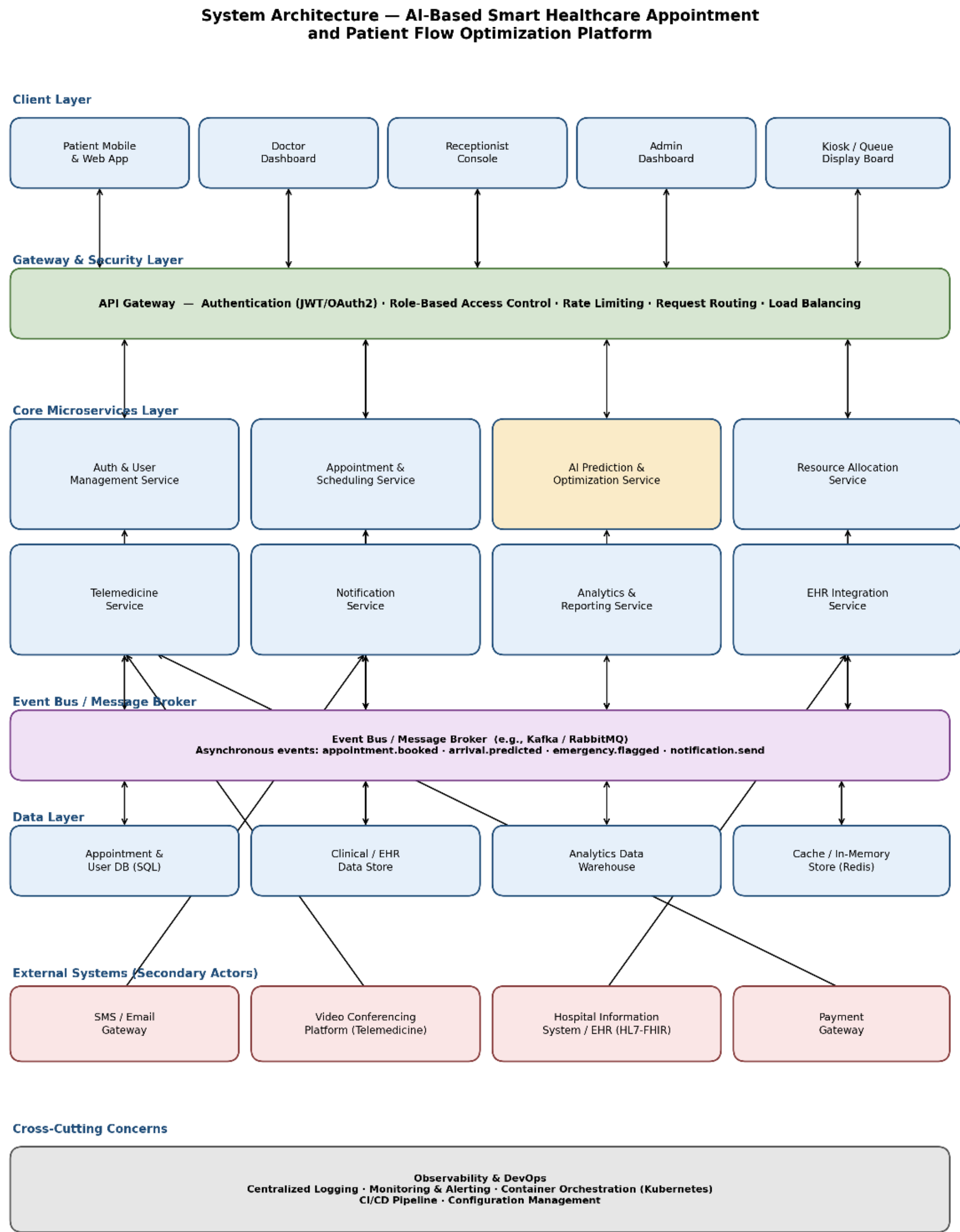


Figure 1: Layered Microservices Architecture — AI-Based Smart Healthcare Appointment and Patient Flow Optimization Platform.

4.1 Diagram Legend and Notation

Element	Meaning
Solid double-headed arrow	Synchronous request/response communication (typically REST over HTTPS).
Rounded rectangle (blue)	An internal application component / microservice.
Rounded rectangle (yellow)	A compute-intensive AI/ML component requiring independent scaling.
Rounded rectangle (purple)	The asynchronous event bus / message broker.
Rounded rectangle (red)	An external, third-party or secondary system.
Rounded rectangle (grey)	Cross-cutting infrastructure and operational tooling.

5. Components and Their Interactions

5.1 Client Layer

The client layer consists of the Patient Mobile&Web App, Doctor Dashboard, Receptionist Console, Admin Dashboard, and Kiosk/Queue Display Board. Each client authenticates through the API Gateway and communicates exclusively over HTTPS; no client accesses microservices or data stores directly.

5.2 Gateway and Security Layer

The API Gateway is the single entry point for all client requests. It performs authentication (JWT/OAuth2) and role-based access control, applies rate limiting to protect backend services, routes requests to the appropriate microservice, and load-balances traffic across service instances. Centralizing these concerns avoids duplicating security logic across every service.

5.3 Core Microservices

Component	Responsibility
Auth&User Management Service	Manages user registration, credentials, roles, and permissions for patients, doctors, receptionists, and administrators.
Appointment&Scheduling Service	Handles appointment booking, rescheduling, and cancellation; maintains the authoritative appointment record.
AI Prediction&Optimization Service	Analyzes historical and real-time data to predict patient arrivals and recommend optimal scheduling/resource decisions.
Resource Allocation Service	Allocates doctors, consultation rooms, and diagnostic resources based on recommendations from the AI service and current queue state.
Telemedicine Service	Manages virtual consultation scheduling and integrates with the external video-conferencing platform.
Notification Service	Sends appointment confirmations, reminders, delay alerts, and queue-status updates via SMS, email, or push notifications.
Analytics&Reporting Service	Aggregates data from other services to produce hospital performance dashboards and periodic reports.
EHR Integration Service	Exchanges patient records and history with the external Hospital Information System using HL7/FHIR standards.

5.4 Event Bus/Message Broker

Services publish domain events (e.g., appointment.booked, arrival.predicted, emergency.flagged, notification.send) to the event bus (e.g., Kafka or RabbitMQ). Interested services subscribe to relevant events, decoupling producers from consumers. This allows, for example, the Notification Service to react to an emergency.flagged event without the Appointment Service needing direct knowledge of the Notification Service.

5.5 Data Layer

Data Store	Used By
Appointment&User DB (SQL)	Auth&User Management Service, Appointment&Scheduling Service
Clinical / EHR Data Store	EHR Integration Service, Telemedicine Service (for consultation history)
Analytics Data Warehouse	Analytics&Reporting Service (read replica / aggregated data, decoupled from live transactional load)
Cache / In-Memory Store (Redis)	Resource Allocation Service and Notification Service, for low-latency queue status and session data

Each core service owns its primary data, following a database-per-service pattern; the Analytics Data Warehouse is populated asynchronously via the event bus rather than by direct queries against operational databases, protecting transactional performance.

5.6 External Systems Integration

- SMS / Email Gateway — invoked by the Notification Service to deliver messages to patients and staff.
- Video Conferencing Platform — invoked by the Telemedicine Service to host remote consultations.
- Hospital Information System / EHR — exchanges patient records with the EHR Integration Service using HL7/FHIR.
- Payment Gateway — invoked for telemedicine consultation fee processing where applicable.

5.7 System Workflow — Illustrative Scenario

Scenario A: Standard Appointment Booking

1. Patient submits a booking request through the mobile app; the request passes through the API Gateway to the Appointment&Scheduling Service.
2. The Appointment Service queries the AI Prediction&Optimization Service for a recommended slot based on predicted demand.
3. On confirmation, the Appointment Service persists the booking and publishes an appointment.booked event to the event bus.
4. The Notification Service consumes the event and sends a confirmation message via the SMS/Email Gateway.

Scenario B: Emergency Case Prioritization

1. Receptionist flags a walk-in patient as an emergency case via the Receptionist Console.
2. The Resource Allocation Service publishes an emergency.flagged event.
3. The Notification Service alerts the relevant doctor and affected patients of queue changes; the Resource Allocation Service recalculates room/doctor assignment in near real time.
4. The Analytics Service asynchronously records the event for later reporting, without impacting the responsiveness of the emergency workflow.

5A. Component Interface Specifications

The table below summarizes representative API endpoints and event contracts exposed by each core microservice. These contracts are the stable boundary other components rely on, allowing internal implementations to change without affecting consumers.

Service	Interface (representative)	Type
Auth&User Management	POST /auth/login, POST /auth/register, GET /users/{id}	REST (sync)
Appointment&Scheduling	POST /appointments, PUT /appointments/{id}, DELETE /appointments/{id}	REST (sync)
Appointment&Scheduling	appointment.booked, appointment.cancelled	Event (async, published)
AI Prediction&Optimization	GET /predict/arrival, GET /optimize/schedule	REST (sync)
AI Prediction&Optimization	arrival.predicted	Event (async, published)
Resource Allocation	GET /resources/availability, POST /resources/assign	REST (sync)
Resource Allocation	emergency.flagged (subscribed), resource.assigned (published)	Event (async)
Telemedicine	POST /telemedicine/sessions, GET /telemedicine/sessions/{id}	REST (sync)
Notification	notification.send (subscribed: appointment.booked, emergency.flagged)	Event (async, subscribed)
Analytics&Reporting	GET /analytics/dashboard, GET /analytics/reports	REST (sync, read-only)
EHR Integration	GET /ehr/patient/{id}, POST /ehr/sync	REST + HL7/FHIR (sync)

All synchronous endpoints are exposed only through the API Gateway and require a valid access token; inter-service calls behind the gateway use mutual TLS and service-to-service authentication.

5B. Deployment View

Each microservice is packaged as an independent container image and deployed on a container orchestration platform (e.g., Kubernetes), allowing independent scaling, versioning, and rollout.

5B.1 Deployment Topology

- Client applications are delivered via a content-delivery network (CDN) for static assets, with API traffic routed to the API Gateway over HTTPS.
- The API Gateway and each microservice run as separate deployments, each with multiple replicas behind a load balancer for redundancy.
- The AI Prediction&Optimization Service is deployed on a separate node pool with higher CPU/GPU allocation, scaled independently based on prediction request volume.
- The event bus (Kafka/RabbitMQ) runs as a managed cluster with multiple broker nodes for durability and throughput.
- Databases are deployed with primary-replica configurations; the Analytics Data Warehouse is populated via asynchronous ETL/event consumption rather than direct writes from client-facing services.
- A staging environment mirrors production for pre-release testing, and blue-green or canary deployment strategies are used to roll out changes with minimal downtime.

5B.2 Environment Tiers

Tier	Purpose
Development	Used by engineers for feature development and unit testing; uses lightweight/mock external integrations.
Staging	Mirrors production configuration for integration and performance testing prior to release.
Production	Serves live hospital traffic with full redundancy, monitoring, and backup in place.

5C. Architectural Risk Assessment

Risk	Impact	Mitigation
AI model produces inaccurate predictions, leading to poor scheduling recommendations.	Reduced trust in the system; suboptimal doctor/resource allocation.	Continuously validate predictions against actual outcomes; allow administrators to override AI recommendations; retrain models on a scheduled basis.
Event bus outage disrupts asynchronous workflows.	Delayed notifications and analytics updates.	Deploy the broker as a redundant cluster; use durable, persisted queues so events are not lost, only delayed.
Increased operational complexity of managing many services.	Higher DevOps overhead and potential for configuration drift.	Use container orchestration with centralized configuration management, logging, and monitoring (Section 5, Cross-Cutting Concerns).
External system (HIS/EHR, video platform) unavailability.	Telemedicine sessions or record sync may fail.	Implement retry logic, circuit breakers, and graceful degradation (e.g., queue EHR sync for later delivery).
Sensitive health data exposure.	Regulatory non-compliance, loss of patient trust.	Enforce encryption at rest/in transit, centralized access control at the gateway, and audit logging of all data access (FR-22).

6. Quality Attribute Analysis

6.1 Scalability

Each microservice can be scaled independently based on load. The AI Prediction&Optimization Service, which is the most compute-intensive, can run on a larger number of instances (or GPU-backed instances) during peak hours without over-provisioning lighter services such as Authentication. Container orchestration (e.g., Kubernetes) enables automatic horizontal scaling based on CPU/queue-length metrics, satisfying NFR-5 (Scalability) from the requirement analysis.

6.2 Security

Authentication and role-based access control are centralized at the API Gateway, ensuring consistent policy enforcement across all clients and services. Sensitive patient data is encrypted in transit (TLS) and at rest, and the EHR Integration Service isolates external HL7/FHIR communication so that internal services are not directly exposed to external systems. This satisfies NFR-2 (Security) and NFR-9 (Data Privacy&Compliance).

6.3 Performance

Asynchronous, event-driven communication prevents slower operations (e.g., analytics aggregation) from blocking time-critical operations (e.g., booking confirmation). A caching layer (Redis) provides low-latency access to frequently read data such as live queue status, helping the AI recommendation and notification workflows meet the 3-second target defined in NFR-1.

6.4 Reliability

Because each service has an independent failure domain, a fault in one service (e.g., the Analytics Service) does not cascade to core booking or emergency-handling functionality. The event bus provides durable message delivery with retry semantics, so a temporarily unavailable Notification Service does not lose emergency alerts or appointment confirmations — they are simply processed once the service recovers.

6.5 Maintainability

The database-per-service pattern and clear service boundaries mean that a change to one capability (for example, adding a new notification channel) requires modifying only the Notification Service, without redeploying unrelated services. Well-defined REST and event contracts between services further reduce the risk of unintended side effects during maintenance.

6.6 Availability

Redundant instances of each service, combined with health checks and automated failover managed by the container orchestration platform, support the 24/7 availability required by NFR-7. Because state is distributed across multiple databases rather than a single monolithic store, a single database outage affects only the corresponding service rather than the entire platform.

7. Architectural Justification and Trade-offs

7.1 Rationale for Chosen Architecture

The hybrid microservices and event-driven architecture was chosen because the system's requirements are naturally decomposable into distinct business capabilities (scheduling, prediction, resource allocation, telemedicine, notifications, analytics, EHR integration) with different scaling, technology, and reliability needs. A single layered or monolithic design would force all these capabilities to scale and deploy together, which conflicts with the requirement that the AI prediction workload scale independently of lighter, high-frequency operations such as authentication.

7.2 Trade-offs Considered

Trade-off	Discussion
Operational complexity vs. flexibility	Microservices introduce additional operational overhead (service discovery, monitoring, orchestration) compared to a monolith, but this is accepted because it enables independent scaling and deployment, which the AI-driven, multi-department use case requires.
Data consistency vs. service autonomy	Database-per-service improves autonomy and fault isolation but requires eventual consistency (e.g., analytics data may lag slightly behind live transactions). This is acceptable because analytics does not require strict real-time consistency, whereas booking and emergency workflows use direct synchronous calls where consistency matters most.
Synchronous vs. asynchronous communication	Synchronous REST calls are used for direct request/response needs (e.g., booking confirmation) for simplicity, while the event bus handles cross-cutting, latency-tolerant workflows (notifications, analytics). A purely event-driven design was rejected because it would complicate simple client-facing request/response interactions.
Build vs. adopt managed services	Using managed message-broker and container-orchestration platforms increases initial learning overhead compared to a simple in-process design, but reduces long-term operational risk and accelerates future scaling.

7.3 Support for Future Enhancements and Long-Term Maintainability

- New capabilities (e.g., a pharmacy-integration service or a patient loyalty/rewards module) can be added as new microservices without modifying existing ones, communicating via the existing event bus and API Gateway.
- The AI Prediction&Optimization Service can be upgraded or replaced (e.g., a new model version) independently, since other services depend only on its published API/event contract, not its internal implementation.
- Additional hospital branches can be onboarded by scaling existing service instances and provisioning additional data partitions, without architectural redesign.
- The clear separation of concerns and documented service contracts make it easier for new developers to understand and maintain individual components, supporting long-term maintainability as the system and team grow.

8. Conclusion

This report analyzed the requirements of the AI-Based Smart Healthcare Appointment and Patient Flow Optimization Platform and selected a hybrid microservices architecture with an API Gateway and event-driven communication as the most suitable design. The architecture diagram, component responsibilities, and interaction workflows were presented, and the design was evaluated against scalability, security, performance, reliability, maintainability, and availability. The chosen architecture balances operational complexity against the flexibility, fault isolation, and independent scalability required by a system that combines real-time scheduling, AI-driven prediction, telemedicine, and hospital-wide analytics, while remaining well positioned for future enhancement and long-term maintenance.